



Camera Sensor 适配指南

文档版本 00B01

发布日期 2025-08-28

版权所有 © 海思技术有限公司2026。保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



HISILICON、海思和其他海思商标均为海思技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受海思公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，海思公司对本文档内容不做任何明示或默示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

海思技术有限公司

地址：上海市青浦区虹桥港路2号101室 邮编：201721

网址：<https://www.hisilicon.com/cn/>

客户服务邮箱：support@hisilicon.com



前言

概述

本文档介绍如何在Camera各SDK版本上开发调试与适配Sensor，通过本文档可以帮助客户在Camera上有序快速的适配新Sensor。

读者对象

本文档（本指南）主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不避免则可能导致轻微或中度伤害的具有低等级风险的危害。
 须知	用于传递设备或环境安全警示信息。如不避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。



修改记录

文档版本	发布日期	修改说明
00B01	2025-08-28	第1次临时版本发布。



目录

前言..... i

1 Sensor 准备活动..... 1

 1.1 准备活动..... 1

2 Sensor 驱动实现..... 3

 2.1 Sensor 库实现..... 3

 2.1.1 Sensor 库结构..... 3

 2.1.2 Sensor 库实例..... 3

 2.1.2.1 实现 Sensor 初始化函数 xxx_init..... 3

 2.1.2.2 实现 cmos_init_ae_exp_function 中 AE 相关回调函数..... 4

 2.1.2.3 实现 cmos_init_sensor_exp_function 中 Sensor 模式相关回调函数..... 5

 2.1.2.4 实现 Sensor 的实例对象 stSnsxxxObj..... 6

 2.1.3 小技巧..... 7

3 适配 Sensor 中间件..... 8

 3.1 新增一款 sensor 配置..... 8

 3.1.1 配置 Sensor 信息..... 8

 3.1.2 注册 Sensor 链表..... 13

 3.1.3 sensor 时钟源、复位源、总线 ID..... 13

 3.2 新增 Sensor 模式..... 14

 3.2.1 添加 Sensor 模式..... 14

 3.3 结构体定义..... 15

 3.3.1 OT_MAPI_SENSOR_ComboDevAttr..... 15

 3.3.2 OT_MAPI_SENSOR_InputAttr..... 16

 3.3.3 OT_MAPI_SENSOR_ModelInfo..... 17

 3.3.4 OT_MAPI_VCAP_SensorMode..... 17

 3.3.5 OT_MAPI_SENSOR_MipiIntf..... 18

 3.3.6 OT_MAPI_SENSOR_CommBusType..... 18

 3.3.7 OT_MAPI_SensorCommCfg..... 19

4 适配 Reference..... 20

 4.1 增加编译配置项..... 20

 4.2 修改配置文件..... 20

 4.3 增加对应参数文件..... 21

 4.4 新增场景自适应参数..... 21



4.5 适配 SDK sysconfig.....	21
4.6 适配 sysconfig 加载参数.....	21
5 常见问题.....	22



插图目录

图 2-1 AE 回调函数.....	4
图 2-2 Sensor 模式回调函数.....	5
图 2-3 实例对象回调函数.....	7



表格目录

表 1-1 重要参数.....	1
表 2-1 cmos_get_ae_default 主要参数.....	4



1 Sensor 准备活动

1.1 准备活动

1. 从正规渠道获取 Sensor datasheet、初始化序列。
2. 查看Sensor datasheet和相关应用手册，特别关注Sensor曝光、增益控制方法，不同模式下曝光和增益的限制，长曝光实现方法，黑电平值，bayer数据输出顺序。
3. 跟Sensor厂家索要所需模式初始化序列，了解各序列的数据数率，Sensor输出总的宽高，精确的帧率是多少等。
4. 确认Sensor控制接口是I2C、SPI还是其他接口，Sensor的设备地址可以通过硬件设置。对于多个摄像头的场景，sensor尽量不要复用I2C总线。不同场景的Sensor IO电平可能是1.8V或者3.3V，设计时需要保证sensor的IO电平与SOC对应的GPIO供电电压一致。如sensor电平是1.8V，则用于sensor控制的GPIO，如Reset，I2C，PRDN，需要使用1.8V供电。
5. 确认Sensor输出数据接口和协议类型，如LVDS、MIPI、并行接口。Sensor 与SOC对接方式详见发布资料包中《xxxx VI 输入场景详细说明》。
6. 确认wdr mode 是VC、DT或者DOL等。
7. 确认不同时序是否需要图像裁剪。

以sensor xxx 1080p90和3840x2160@30为例，这些重要参数，如表1-1所示。

表 1-1 重要参数

Rawsize H	RawSize V	Frame_lengt h_lines	Line_lengt h_pck	Data rate [Mbps/ lane]	Lane	INCLK [MHz]	FPS	BIT
1920	1080	1108	5192	1296.00	4	24	90.11	10
3840	2160	3228	5352	1296.00	4	24	30.00	10



须知

以上信息参考Sensor厂商提供的技术手册，包括XXX_RegisterSetting、XXX_Module_Design_Reference_Manual、XXX_Data_Sheet。



2 Sensor 驱动实现

拿到一款新的Sensor后，首先要做的是在MPP层，通过添加Sensor驱动和库，配置MIPI、VI模块属性，保证最基本的sample_vio可以正常执行并正常出图。

本章节重点描述Sensor驱动和库的实现，也即生成Sensor库，其他模块的配置，请根据Sensor的实际情况，参考MPP下各模块的使用说明进行配置。

2.1 Sensor 库实现

2.1.1 Sensor 库结构

Sensor库结构，一般包括4个文件：xxx_cmos.c、xxx_sensor_ctl.c、xxx_cmos_priv.h、xxx_cmos_ex.h。

- xxx_cmos.c包含Sensor驱动的主要功能函数，里面实现Sensor相关的AE、AWB、ISP默认配置、Sensor启动模式选择函数、Sensor向AE、AWB、ISP注册和反注册函数、sensor库实例对象（g_sns_xxx_obj）函数；
- xxx_sensor_ctl.c主要包含Sensor的初始化序列、通讯接口初始化、读写函数实现；
- xxx_cmos_priv.h是一些结构、模式类型等定义的头文件，该组成部分有时不独立成一个文件；
- xxx_cmos_ex.h主要是ISP各算法模块的模式配置，有时这部分未独立成一个文件，分布在xxx_cmos.c中。

2.1.2 Sensor 库实例

了解了Sensor库的组成，下面以sensor xxx的3840x2160@30序列为例，详细说明主要函数的实现。

2.1.2.1 实现 Sensor 初始化函数 xxx_init

- 根据Sensor手册和硬件连接了解到，该Sensor是I2C通讯，器件的设备地址为0x34，因此先基于I2C驱动，完成xxx_i2c_init、xxx_i2c_exit、xxx_read_register、xxx_write_register这四个I2C操作基本接口。
- 根据Sensor厂家提供的初始化序列，实现xxx_init。注意有些Sensor对配置初始化寄存器要求较高，会有顺序、延时的限制，因此需要严格按照Sensor厂家的逻辑



实现。请参考Sensor厂家提供Sensor datasheet里的Power ON sequence相关章节。

2.1.2.2 实现 cmos_init_ae_exp_function 中 AE 相关回调函数

图2-1为sensor xxx cmos_init_ae_exp_function中需要设置的回调函数，WDR相关函数及其详细描述请参考《ISP开发参考》。

图 2-1 AE 回调函数

```
pstExpFuncs->pfn_cmos_get_ae_default = cmos_get_ae_default; //AE默认配置
pstExpFuncs->pfn_cmos_fps_set = cmos_fps_set; //设置帧率
pstExpFuncs->pfn_cmos_slow_framerate_set = cmos_slow_framerate_set; //设置降帧，长曝光用此函数实现
pstExpFuncs->pfn_cmos_inttime_update = cmos_inttime_update; //设置曝光时间
pstExpFuncs->pfn_cmos_gains_update = cmos_gains_update; //设置增益
pstExpFuncs->pfn_cmos_again_calc_table = cmos_again_calc_table; //模拟增益计算
pstExpFuncs->pfn_cmos_dgain_calc_table = cmos_dgain_calc_table; //数字增益计算
```

- **cmos_get_ae_default**，这个函数主要设置针对Sensor AE相关的一些设置，主要是Again、Dgain、曝光时间的最大最小值，还有当前模式下Sensor的总行数full_lines_std，以及每500ms多少行lines_per500ms，从Sensor手册获取到这些参数的限制。
 - 设置Again、Dgain的计算方法accu_type，cmos_again_calc_table，cmos_dgain_calc_table这两个函数要根据accu_type设置的计算方法计算sensor应该配置的Again、Dgain寄存器应该配置的值。
 - 当使用快速启动时，初始曝光量init_exposure和lines_per500ms要用对应全局变量赋值，否则快速启动功能异常。其他参数请参考《ISP开发参考》关于ot_isp_ae_sensor_default的描述。

表 2-1 cmos_get_ae_default 主要参数

full_lines_std	max_again	max_dgain	max_int_time_target	min_again/min_dgain	min_int_time
1920	1080	1108	5192	1296.00	4
3840	2160	3228	5352	1296.00	4

- **cmos_again_calc_table**，该函数用来实现AE所使用的Again和sensor寄存器配置的Again的换算关系。

AE算法内部使用的是带有10bit精度的倍数，即1024代表1倍增益，而Sensor内部使用的增益的单位可能是DB，也可能是不同精度的倍数，因此两者之间需要做转换。

以Sensor xxx为例：内部生效的Again=512/（512-x），x是sensor寄存器配置的值。那么通过AE计算的Again就能计算出Sensor寄存器应该配置的值x=512-512/Again。注意这里Again是指实际倍数，1倍代表0DB。实际应用中，AE计算的Again是10bit精度的整形，因此要做精度转换，那么x=512-(512<<10)/Again。

 - 函数形参*again_lin作为输入代表AE当前的计算结果，作为输出代表经过此函数计算实际配给Sensor的倍数，比如AE计算当前应该配给Sensor的Again是

4567，但是由于Sensor Again精度没有那么高（这里只是打比方，Sensor的Again精度还是比较高的），只能生效4096，那么此函数在返回时，要重新给*again_lin赋值4096，告诉AE，实际配给Sensor的Again是4096而非4567，需要AE重新分配剩余部分。

- 函数形参*again_db代表计算完的应该在cmos_gains_update函数中用到的配给Sensor寄存器的值。
- **cmos_dgain_calc_table**，计算Sensor Dgain，实现方法和Again一样。
- **cmos_gains_update**，将计算好的Again、Dgain给Sensor对应的寄存器赋值。
- **cmos_inttime_update**，用AE计算好的曝光时间给Sensor对应的寄存器赋值，这里要注意的是函数参数int_time是以行为单位的。
- **cmos_fps_set**，设置帧率，通过pub属性更改Sensor帧率就是调用此函数实现。即通过ss_mpi_isp_set_pub_attr接口改变Sensor帧率。当前改变Sensor输出帧率通常是通过改变Sensor输出的垂直消隐区长度，即增加或者减少Sensor的行数。根据输入fps的目的帧率，结合当前Sensor模式下的标称帧率和总行数（指初始化序列定义的帧率和总行数）计算出目的帧率对应的总行数值，并更新AE默认参数ae_sns_dft->fps、ae_sns_dft->lines_per500ms、ae_sns_dft->full_lines_std、ae_sns_dft->full_lines、ae_sns_dft->max_int_time、以及sensor上下文里的全局变量sns_state->fl_std、sns_state->fl[0]。更新sensor对应的寄存器。
- **cmos_slow_framerate_set**，实现自动降帧函数，拍照时的长曝光就是通过此函数实现。和设置帧率函数类似，此函数也是通过改变Sensor的消隐区长度改变帧率，只是这里AE已经计算好了期望的行数，函数实现的只是根据Sensor总行数限制和实现长曝光的方法，给Sensor寄存器赋值。对Sensor来说，和设置帧率不同的是长曝光还有另一个寄存器CIT_LSHIFT控制，该寄存器可以使曝光时间最长达到COARSE_INTEG_TIME寄存器所设曝光时间的128倍，因此需要分两种情况：
 - 小于或等于COARSE_INTEG_TIME寄存器最大值
 - 大于COARSE_INTEG_TIME寄存器最大值
 分别有不同的赋值方法，详细赋值可以参考Sensor手册里长曝光说明。
 注意除了给Sensor寄存器赋值外，还需要更新AE默认配置ae_sns_dft->full_lines_std、ae_sns_dft->max_int_time和sensor上下文里的全局变量sns_state->fl[0]。

2.1.2.3 实现 cmos_init_sensor_exp_function 中 Sensor 模式相关回调函数

Sensor模式回调函数，如图2-2所示。

图 2-2 Sensor 模式回调函数

```
sensor_exp_func->pfn_cmos_sns_init ..... = xxx_init;
sensor_exp_func->pfn_cmos_sns_exit ..... = xxx_exit;
sensor_exp_func->pfn_cmos_sns_global_init ..... = sensor_global_init;
sensor_exp_func->pfn_cmos_set_image_mode ..... = cmos_set_image_mode;
sensor_exp_func->pfn_cmos_set_wdr_mode ..... = cmos_set_wdr_mode;
sensor_exp_func->pfn_cmos_get_isp_default ..... = cmos_get_isp_default;
sensor_exp_func->pfn_cmos_get_isp_black_level ..... = cmos_get_isp_black_level;
sensor_exp_func->pfn_cmos_get_blc_clamp_info ..... = TD_NULL;
sensor_exp_func->pfn_cmos_set_pixel_detect ..... = cmos_set_pixel_detect;
sensor_exp_func->pfn_cmos_get_sns_reg_info ..... = cmos_get_sns_regs_info;
```



- **xxx_init**, Sensor初始化序列配置函数实现, 在第**Sensor准备活动**章节已经说明。
- **xxx_exit**, Sensor退出函数, 实际未对Sensor进行任何操作, 只是将I2C设备退出和一些全局变量清零。
- **sensor_global_init**, Sensor上下文全局变量初始化。
- **cmos_set_image_mode**, Sensor模式选择函数。此函数会根据pub属性设置的sns_size.width、sns_size.height、wdr_mode、sns_mode选择不同的初始化模式。
需要注意: 通常情况下, 优先通过分辨率和WDR模式来选择不同序列, 而相同分辨率的不同帧率、bit位下的不同序列, 通过u8SnsMode区分。
- **cmos_set_wdr_mode**, 根据不同WDR模式设置不同序列所对应的一些全局变量sns_state->wdr_mode、sns_state->fl_std。
- **cmos_get_isp_default**, 设置不同模式下各ISP算法的默认配置, 未经调优时可以不配置, 调优之后再根据调试的最优参数在这里添加算法的默认参数。
- **cmos_get_isp_black_level**, 这是Sensor的黑电平, 如果不同sensor增益下黑电平有漂移, 可以将update设置成真, 根据不同的增益设置不同的黑电平, 通常情况黑电平漂移很小, 可以只设一组固定值。
- **cmos_set_pixel_detect**, 静态坏点标定时对Sensor的设置, 通常情况下将帧率设置到5fps, 曝光时间设为最长, 增益设为最小, 此时有问题的像素点才被当作坏点。
- **cmos_get_sns_regs_info**, 设置Sensor寄存器函数, 此函数设置在ISP运行过程中, 都有哪些Sensor寄存器会被配置, 延时关系是怎样的。

详细说明请参考《ISP开发参考》中关于ot_isp_sns_regs_info结构体的描述。这里重点是配置延时关系, reg_num是指AE计算完第几帧开始配置此寄存器, cfg2_valid_delay_max是指从配置Sensor寄存器到寄存器值生效开始输出对应的RAW数据的最大帧数。根据Sensor应用手册配置, 比如sensor xxx, 参考Sensor厂家提供的应用手册中的Mode Transition Sequence and Register Update Timing章节, sensor曝光时间、增益和总行数寄存器都可以同时配置无需对哪个做延时, 因此reg_num都设置为0, 从配置Sensor到寄存器值生效开始输出RAW的最大延时帧数是两帧, 则cfg2_valid_delay_max设置为2。

2.1.2.4 实现 Sensor 的实例对象 stSnsxxxObj

定义指向Sensor的实例对象g_sns_xxx_obj, 应用层可以使用此实例对象直接回调Sensor的一些控制函数。



图 2-3 实例对象回调函数

```
ot_isp_sns_obj g_sns_xxx_obj = {  
    .pfn_register_callback = cmos_register_callback,  
    .pfn_unregister_callback = cmos_unregister_callback,  
    .pfn_standby = cmos_standby,  
    .pfn_restart = cmos_restart,  
    .pfn_mirror_flip = TD_NULL,  
    .pfn_set_blc_clamp = TD_NULL,  
    .pfn_write_reg = cmos_write_register,  
    .pfn_read_reg = cmos_read_register,  
    .pfn_set_bus_info = cmos_set_bus_info,  
    .pfn_set_init = cmos_set_init,  
    .pfn_set_fast_ae = cmos_set_fast_ae  
};
```

如果没有实现的功能，可以给指针赋空。详情请参考《ISP开发参考》中ot_isp_sns_obj描述。适配完这些函数后，需要在ot_sns_ctrl.h中将此对象做外部声明。

2.1.3 小技巧

跑业务如果遇到不出图时的一些定位小技巧，操作如下：

- 步骤1** 在xxx_cmos.c里，先把cmos_get_sns_regs_info函数直接返回，也就是对Sensor只执行初始化序列的配置，运行过程中不改Sensor任何寄存器，因为如果Sensor曝光、宽高不满足限制会导致Sensor输出异常，因此第一步先保证序列没问题。
- 步骤2** 保证Sensor序列配置成功之后（可以通过i2c_read工具查看Sensor寄存器是否配置成功）如果还是没有输出，要以从前到后的顺序依次排查MIPI、VI、ISP、VO是否输出正常，其配置是否正确。
- 步骤3** 基本序列正常后再将cmos_get_sns_regs_info正常配置，进一步确认AE相关配置是否正确，如果正确AE就可以根据场景的变化调整曝光了。
- 步骤4** AE正常之后再进一步确认其他函数功能是否正常。

到此，Sensor驱动部分就完成了。下一步就可以开始优化图像效果和进入整机适配流程了。

----结束



3 适配 Sensor 中间件

MPP Sensor驱动和库开发完成之后，接下来就是在中间件中适配相关流程，以便应用APP更好的开、关，切换时序等。

3.1 新增一款 sensor 配置

3.1.1 配置 Sensor 信息

新增一款Sensor配置Sensor相关信息步骤如下：

步骤1 增加新Sensor的宏定义

在ndk/code/include/ot_mapi_vcap_define.h头文件中，找到OT_MAPI_VCAP_SensorType枚举类型，并增加新增Sensor的枚举。

以hy_s0603为例：OT_MAPI_VCAP_SENSOR_HY_S0603

步骤2 增加新Sensor对应的文件

在ndk/code/mediaserver/configs/sensor/\$(chip)目录下新建一个该Sensor的文件夹，文件夹下新增sensor_interface_cfg_params.c和Makefile两个文件，其中\$(chip)的取值支持hi3516dv500、ss928v100以及hi3516cv610。

说明

当使用的芯片为Hi3516DV500或者Hi3519DV500时，\$(chip)取值为hi3516dv500；当使用SS927V100或者SS928V100时，\$(chip)取值为ss928v100。

生产的库名称按如下格式，例如hy_s0603的Makefile：

```
TARGET := ss_mapi_cfg_hy_s0603
```

步骤3 配置OT_MAPI_SENSOR_ComboDevAttr结构体

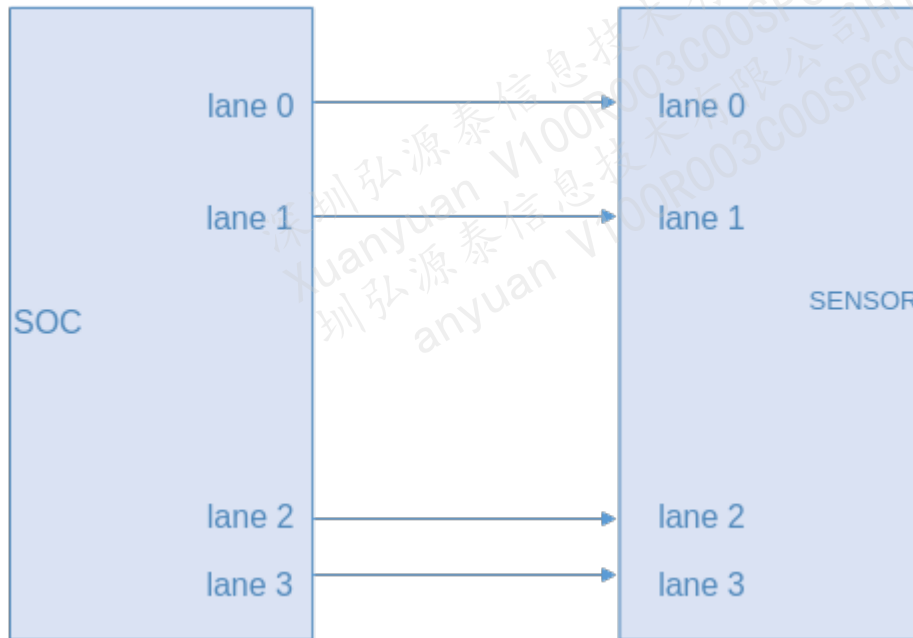
在ndk/code/mediaserver/configs/sensor/\$(chip)/sensor_xx/sensor_interface_cfg_params.c文件配置OT_MAPI_SENSOR_ComboDevAttr结构体。

data_rate：一般情况设为MIPI_DATA_RATE_X1，如果Sensor模式有大于4K60fps的，就可设为MIPI_DATA_RATE_X2。



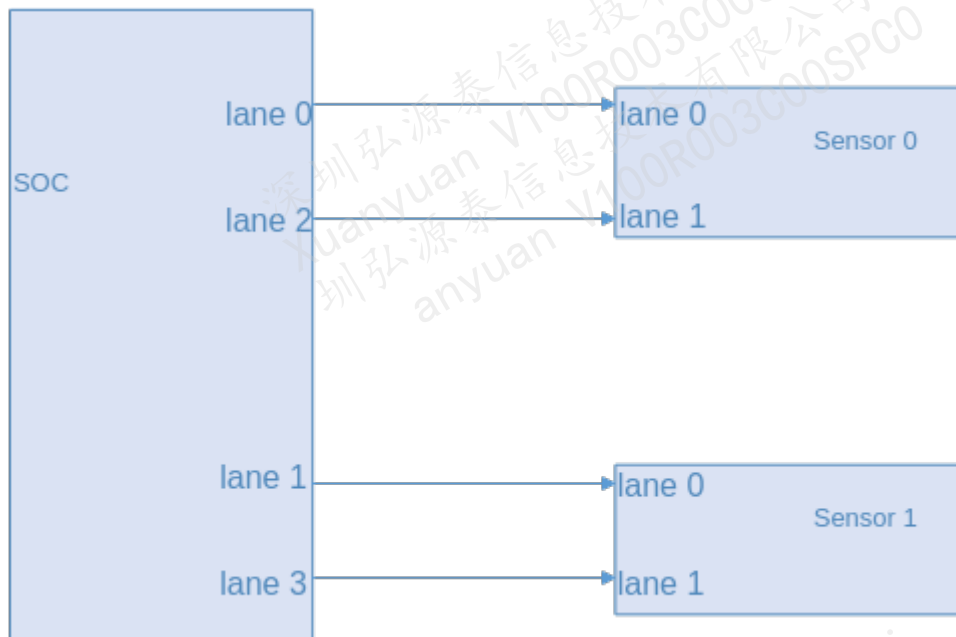
aLaneId配置由硬件单板与实际Sensor输出通道的对应关系决定，详情请参考所使用芯片SDK发布包中的《MIPI使用指南》，常见路径为01.software\board\MPP下。LaneId的配置与硬件单板，sensor以及sensor所连接的mipi_dev位置有关，此外不同时序使用的lane个数也可能会有差异。

在LANE_DIVIDE_MODE_0配置下，lane的连线图：



在这种连线图下，lane数组配置就是{0, 1, 2, 3}；如实际硬件连线与此不符，则按实际硬件连线配置。

在LANE_DIVIDE_MODE_1配置下，lane的连线图：





在这种连线图下，sensor 0 lane数组配置就是{0, 2, -1, -1}; sensor 1 lane数组配置就是{1, 3, -1, -1}。具体请参考《MIPI使用指南》中mipi_dev_attr_t结构体的描述。

以hy_s0603为例说明各参数的配置：

```
static OT_MAPI_SENSOR_CombosDevAttr g_sensorCfg = {
/* config SENSOR */
.sensorObj = &g_sns_hy_s0603_obj,
.sensorType = OT_MAPI_VCAP_SENSOR_HY_S0603,
.inputMode = INPUT_MODE_MIPI,
.dataRate = MIPI_DATA_RATE_X1,
.sensorI2cAddr = 0x60,
.sensorInputAttr = {
.bayerFormat = OT_ISP_BAYER_BGGR,
.sensorCommBusType = OT_SENSOR_COMMBUS_TYPE_I2C,
},
.sensorModeInfo = {
.sensorModeCnt = sizeof(g_sensorMode) / sizeof(OT_MAPI_VCAP_SensorMode),
.sensorMode = (void *)g_sensorMode,
.mipiIntf = (void *)g_sensorMipiIntfCfg
},
.laneId = {
#if defined CFG_LANE_DIVIDE_MODE_0
/* mipi_dev0 */
{
{0, 1, 2, 3, -1, -1, -1, -1}, /* 4K30 */
{0, 1, 2, 3, -1, -1, -1, -1}, /* 4K30 wdr */
},
},
},
},
},
},
},
/* No mipi_dev1 */
{
}, {0, 0, 0, 0, 0, 0},
},
/* No mipi_dev2 */
{
}, {0, 0, 0, 0, 0, 0},
},
/* No mipi_dev3 */
{
}, {0, 0, 0, 0, 0, 0},
},
#elif defined CFG_LANE_DIVIDE_MODE_1
/* mipi_dev0 */
{
{0, 1, 2, 3, -1, -1, -1, -1}, /* 4K30 */
{0, 1, 2, 3, -1, -1, -1, -1}, /* 4K30 wdr */
},
},
},
},
},
/* mipi_dev1 */
{
}, {0, 0, 0, 0, 0, 0},
},
/* mipi_dev2 */
{
{4, 5, 6, 7, -1, -1, -1, -1}, /* 4K30 */

```



```

{4, 5, 6, 7, -1, -1, -1, -1}, /* 4K30 wdr */
},
},
},
},
},
},
/* mipi_dev3 */
{
}, {}, {}, {}, {}, {},
},
#elif defined CFG_LANE_DIVIDE_MODE_2
/* mipi_dev0 */
{
{0, 1, 2, 3, -1, -1, -1, -1}, /* 4K30 */
{0, 1, 2, 3, -1, -1, -1, -1}, /* 4K30 wdr */
},
},
},
},
},
},
/* mipi_dev1 */
{
}, {}, {}, {}, {}, {},
},
/* mipi_dev2 */
{
{4, 6, -1, -1, -1, -1, -1, -1}, /* 4K30 */
{4, 6, -1, -1, -1, -1, -1, -1}, /* 4K30 wdr */
},
},
},
},
},
},
/* mipi_dev3 */
{
{5, 7, -1, -1, -1, -1, -1, -1}, /* 4K30 */
{5, 7, -1, -1, -1, -1, -1, -1}, /* 4K30 wdr */
},
},
},
},
},
},
#endif
},
};

```

步骤4 Sensor时序与Sensor模式

OT_MAPI_MIPI_INTF_S、OT_MAPI_VCAP_SensorMode参考[结构体定义](#)章节描述。

用户sensor时序选择：OT_MAPI_VCAP_InitSensor接口传下来的参数会和OT_MAPI_VCAP_SensorMode g_stSensorMode [] 数组下的成员匹配，匹配成功后返回数组下标，g_stSensorMipiIntfCfg数组和g_stSensorMode数组是一一对应的，根据下标就可找到g_stSensorMipiIntfCfg数组对应的属性配置。

以hy_s0603为例说明各模式的配置：

```

static const OT_MAPI_SENSOR_MipiIntf g_sensorMipiIntfCfg[] = {
/* config 4k@30fps */
{
    .imgRect = {
        .x = 0,

```



```
.y = 0,
.width = 3840,
.height = 2160
},
.snsSize = {
.width = 3840,
.height = 2160
},
.mipiAttr = {
.input_data_type = DATA_TYPE_RAW_12BIT,
.wdr_mode = OT_MIPI_WDR_MODE_NONE,
},
},
/* config 4k@30fps wdr */
{
.imgRect = {
.x = 0,
.y = 0,
.width = 3840,
.height = 2160
},
.snsSize = {
.width = 3840,
.height = 2160
},
.mipiAttr = {
.input_data_type = DATA_TYPE_RAW_12BIT,
.wdr_mode = OT_MIPI_WDR_MODE_VC,
},
},
};
/*sensor支持的各个模式*/
static const OT_MAPI_VCAP_SensorMode g_sensorMode[] = {
/* SENSOR_MODE_4K_30FPS frameRate:[1.0-30] */
{
.width = 3840,
.height = 2160,
.snsMode = 0,
.wdrMode = OT_WDR_MODE_NONE,
.snsMaxFrameRate = 30.0f,
.mipiExtData = &g_mipiExtDataTypeHY_S0603NowdrAttr
},
{
.width = 3840,
.height = 2160,
.snsMode = 0,
.wdrMode = OT_WDR_MODE_2To1_LINE,
.snsMaxFrameRate = 30.0f,
.mipiExtData = NULL
},
},
};
```

步骤5 配置sensor的初始化函数

以hy_s0603为例，在sensor_interface_cfg_params.c文件下面添加SensorHY_S0603Init()函数，把所配置的sensor信息注册到链表中去，以便vcap接口调用。

g_sensorCfg即为用户配置的sensor信息结构体。

```
TD_U32 SensorHY_S0603Init(void)
{
```



```
return MAPI_SENSOR_Register((OT_MAPI_SENSOR_ComboDevAttr *)&g_sensorCfg);
}
```

在ndk/code/mediaserver/configs/sensor/include/sensor_interface_cfg_params.h文件中增加Sensor初始化函数声明，如

```
TD_U32 SensorHY_S0603Init(void);
```

----结束

3.1.2 注册 Sensor 链表

在ndk/code/mediaserver/configs/sensor/(chip)/comm/register_sensor/目录下register_sensor.c文件中调用MAPI_SENSOR_Load 函数注册对应sensor库。

以hy_s0603为例：

```
#ifdef SENSOR_HY_S0603
CHECK_MAPI_VCAP_RET(SensorHY_S0603Init(), "SensorHY_S0603 param load fail!!!");
MAPI_INFO_TRACE(OT_MAPI_MOD_VCAP, "SensorHY_S0603 param load success!\n");
#endif
```

该宏的名称为SENSOR_HY_S0603，定义位置位于ndk/Makefile.param中，通过CFLAGS += \$(foreach SENSOR_TYPE,\$(CFG_SENSOR_LIST), -DSENSOR_\$ (SENSOR_TYPE))语句，由SENSOR_字符串拼接\$(SENSOR_TYPE)变量定义，SENSOR_TYPE则由CFG_SENSOR_LIST决定。

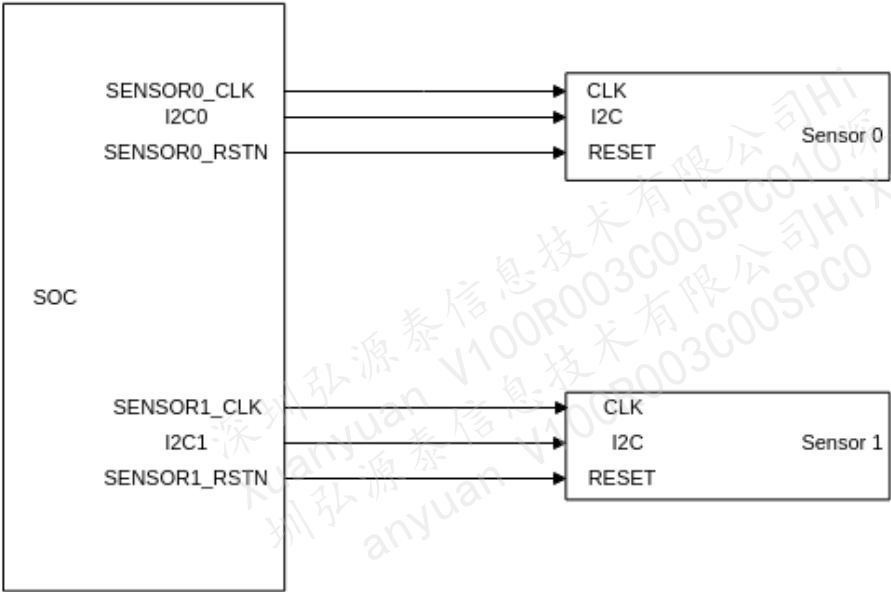
CFG_SENSOR_LIST的默认值定义于config_hi3516dv500.mak、config_ss928v100.mak、config_hi3516cv610.mak文件中，支持编译时覆盖。SensorHY_S0603Init()函数是步骤5中注册到链表的函数。

3.1.3 sensor 时钟源、复位源、总线 ID

Sensor时钟源、复位源和总线ID是在ndk/code/mediaserver/configs/sensor/(chip)/comm/sensor_intf_comm_params/comm_sensor_intf_cfg_params.c文件中OT_MAPI_SENSOR_COMM_CFG_S g_allCommSnsIntfCfg结构体变量中配置的，需要根据实际连线来配置。

SOC提供两个sensor时钟源，分别为：SENSOR_CLK0、SENSOR_CLK1、编号分别为0、1。用户需要根据实际的硬件连线去配置每个Sensor的时钟源，没有用到的写-1。

sensor连线示例说明：



这种连线情况sensor0，sensor1时钟源分别为0、1，复位源分别为0、1，busId对应I2C的连线，也为0、1。

3.2 新增 Sensor 模式

3.2.1 添加 Sensor 模式

- 增加新时序需要在文件sensor_interface_cfg_params.c中，**g_stSensorMode []**数组最后新增一个sensor模式，同时在**g_stSensorMipiIntfCfg []**数组最后新增加时序对应的属性配置。
- 在**OT_MAPI_SENSOR_ComboDevAttr**结构体aLaneId三维数组中，把该时序对应的Lane id配置添加到每个Lane的分布模式、每个mipi dev下。

以HY_S0603为例增加一个新模式1080P@30fps_wdr:

```
static const OT_MAPI_MIPI_INTF_S g_stSensorMipiIntfCfg [] =
{
.....
/* config 1080P@30fps_WDR */
{
    .img_rect = {
        .x = 0,
        .y = 0,
        .width = 1920,
        .height = 1080
    },
    .stSnsSize = {
        .u32Width = 1920,
        .u32Height = 1080
    },
    .mipi_attr =
    {
        .input_data_type = DATA_TYPE_RAW_10BIT,
        .wdr_mode = OT_MIPI_WDR_MODE_DOL,
```



```
    },
},
};

static const OT_MAPI_SENSOR_MODE_S g_stSensorMode [] =
{
.....
/* SENSOR_MODE_1080p_30FPS_WDR frameRate:[1.0-30] */
{
    .width = 1920,
    .height = 1080,
    .snsMode = 0,
    .wdrMode = WDR_MODE_2To1_LINE,
    .snsMaxFrameRate = 30.0f
},
};
};
```

3.3 结构体定义

- **OT_MAPI_SENSOR_ComboDevAttr**: 用来配置mipi和isp的属性。
- **OT_MAPI_SENSOR_InputAttr**: sensor输入属性配置。
- **OT_MAPI_SENSOR_ModeInfo**: sensor模式相关信息。
- **OT_MAPI_VCAP_SensorMode**: sensor模式配置信息。
- **OT_MAPI_SENSOR_MipiIntf**: mipi时序信息。
- **OT_MAPI_SENSOR_CommBusType**: sensor连线方式。
- **OT_MAPI_SensorCommCfg**: sensor公共配置信息。

3.3.1 OT_MAPI_SENSOR_ComboDevAttr

【定义】

```
typedef struct {
    void*                sensorObj;
    TD_S32               sensorType;
    input_mode_t         inputMode;
    mipi_data_rate_t     dataRate;
    unsigned char        sensorI2cAddr;OT_MAPI_SENSOR_InputAttr
    sensorInputAttr;OT_MAPI_SENSOR_ModeInfosensorModeInfo;
    short laneId[OT_MAPI_VCAP_MAX_DEV_NUM][OT_MAPI_SENSOR_MAX_MODE_NUM]
[OT_MAPI_SENSOR_MAX_LANE_NUM];
    TD_VOID *libHandle;struct cbb_list_head    list;
} OT_MAPI_SENSOR_ComboDevAttr;
```

【成员】

成员名称	描述
sensorObj	Sensor回调函数，MPP头文件提供，注册给ISP用。定义在ot_sns_ctrl.h文件中。 对于使用AHD sensor，这里值不用填。



成员名称	描述
sensorType	头文件中定义的Sensor类型的宏，定义在sensor_interface_cfg_params.h文件中。 对于使用AHD sensor，这里一般配置为BT.656。
inputMode	接口输入类型，如INPUT_MODE_MIPI、INPUT_MODE_SLVS、INPUT_MODE_LVDS、INPUT_MODE_BT656；参考 准备活动 。
dataRate	MIPI Rx、SLVS 输入速率，为MIPI_DATA_RATE_X1或MIPI_DATA_RATE_X2。
sensorI2cAddr	Sensor的I2C地址
sensorInputAttr	Sensor输入属性。 对于使用AHD sensor，这里值不用填。
sensorModelInfo	Sensor各模式配置信息。
laneId	Lane id配置。详情请参考《MIPI使用指南》1.3小节。 对于使用AHD sensor，当使用BT.656协议接入时，这里可不用填。
libHandle	动态加载sensor库的句柄文件。
list	链表节点。Mapi内部赋值，用户无需关注。

【注意】

无

3.3.2 OT_MAPI_SENSOR_InputAttr

【定义】

```
typedef struct {  
    ot_isp_bayer_format bayerFormat;  
    OT_MAPI_SENSOR_CommBusType sensorCommBusType;  
} OT_MAPI_SENSOR_InputAttr;
```

【成员】

成员名称	描述
bayerFormat	Bayer域图像输出顺序，定义参考《ISP开发参考》文档，其值参考 准备活动 章节。
sensorCommBusType	Sensor控制接口是spi还是i2c，参考 准备活动 章节。

【注意】



Bayer域图像输出顺序参考Sensor datasheet，如果用户在sensor驱动中对图像做了裁剪或者旋转，那么默认的图像输出顺序也会对应改变。

3.3.3 OT_MAPI_SENSOR_ModelInfo

【定义】

```
typedef struct {
    TD_S32 sensorModeCnt;
    const OT_MAPI_VCAP_SensorMode *sensorMode;
    const OT_MAPI_SENSOR_MipiIntf *mipiIntf;
} OT_MAPI_SENSOR_ModelInfo;
```

【成员】

成员名称	描述
sensorModeCnt	该sensor支持的sensor模式个数，其值为sensorMode与mipiIntf数组变量的个数。
sensorMode	Sensor模式数组。
mipiIntf	Sensor时序数组。

【注意】

无

3.3.4 OT_MAPI_VCAP_SensorMode

【定义】

```
typedef struct {
    TD_S32 width; /* it is related to mipi width */
    TD_S32 height; /* it is related to mipi height */
    OT_MAPI_VCAP_WdrMode wdrMode;
    TD_U32 snsMode;
    TD_FLOAT snsMaxFrameRate;
    OT_MAPI_VCAP_ExtData *mipiExtData;
} OT_MAPI_VCAP_SensorMode;
```

【成员】

成员名称	描述
width	对应OT_MAPI_SENSOR_MipiIntf中img_rect的宽度值。
height	对应OT_MAPI_SENSOR_MipiIntf中img_rect的高度值。
wdrMode	Wdr模式，对应参考“准备活动”。
snsMode	Sensor模式，对应参考“Sensor驱动实现”。
snsMaxFrameRate	最大帧率，对应参考表1-1的FPS。



【注意】

无

3.3.5 OT_MAPI_SENSOR_MipiIntf

【定义】

```
typedef struct {  
    img_rect_t imgRect; /* it is related to mipi image_rect */  
    OT_MAPI_Size snsSize; /* it is related to sensor SIZE_S */  
    union {  
        mipi_dev_attr_t mipiAttr;  
        lvds_dev_attr_t lvdsAttr;  
    };  
} OT_MAPI_SENSOR_MipiIntf;
```

【成员】

成员名称	描述
imgRect	Mipi输出宽高，可用户黑边裁剪，或者图像大小裁剪；若用于黑边裁剪，裁剪大小参考“ 准备活动 ”
snsSize	Sensor输出宽高，用于ISP_PUB_ATTR_S属性的sensor宽高；其值对应准备活动章节 表1-1 的RawsizeV、RawSizeH
mipiAttr	Mipi属性的定义参考《MIPI使用指南》。只需设置其中的input_data_type与wdr_mode。 wdr_mode其值参考 准备活动 ”。 Input_data_type其值参考 表1-1 的BIT。 对于AHD sensor，这里值可不填。
lvdsAttr	lvds_dev_attr_t请参考《MIPI使用指南》

【注意】

无

3.3.6 OT_MAPI_SENSOR_CommBusType

【定义】

```
typedef enum {  
    OT_SENSOR_COMMBUS_TYPE_I2C = 0,  
    OT_SENSOR_COMMBUS_TYPE_SPI,  
    OT_SENSOR_COMMBUS_TYPE_BUTT  
} OT_MAPI_SENSOR_CommBusType;
```

【成员】

成员名称	描述
OT_SENSOR_COMMBUS_TYPE_I2C	I2C连线方式



成员名称	描述
OT_SENSOR_COMMBUS_TYPE_SPI	Spi连线方式

【注意】

无

3.3.7 OT_MAPI_SensorCommCfg

【定义】

```
typedef struct {
    lane_divide_mode_t laneIdMode;
    sns_clk_source_t snsClkSource[OT_MAPI_VCAP_MAX_DEV_NUM];
    sns_rst_source_t snsRstSource[OT_MAPI_VCAP_MAX_DEV_NUM];
    TD_S32 busId[OT_MAPI_VCAP_MAX_DEV_NUM];
} OT_MAPI_SensorCommCfg;
```

【成员】

成员名称	描述
laneIdMode	Lane模式配置；内部自动赋值
snsClkSource	Sensor时钟信号线编号， OT_MAPI_VCAP_MAX_DEV_NUM定义请参考《MAPI V1.0 媒体处理开发参考》
snsRstSource	Sensor复位线编号，OT_MAPI_VCAP_MAX_DEV_NUM 定义请参考《MAPI V1.0 媒体处理开发参考》
busId	Sensor控制线，参考“ 准备活动 ”， OT_MAPI_VCAP_MAX_DEV_NUM定义请参考《MAPI V1.0 媒体处理开发参考》

【注意】

无



4 适配 Reference

4.1 增加编译配置项

新增一块新Sensor时，需要在HiXuanyuan代码中进行编译配置项的增加，以HY_S0603为例：

在HiXuanyuan/cmake/build/base.mak中，增加

```
else ifeq ($(CONFIG_SENS0_HY_S0603), y)
NDK_COMPILE_SENSOR_LIST += HY_S0603
```

在HiXuanyuan/cmake/build/kconfig.mak中，增加

```
else ifeq ($(CONFIG_SENS0_HY_S0603), y)
CFG_SENSOR_TYPE0 := hy_s0603
KCONFIG_CFLAGS += -DCONFIG_SENS0_HY_S0603
```

在HiXuanyuan/cmake/menuconfig/Kconfig.board中，增加

```
config SENS0_HY_S0603
bool "hy_s0603"
depends on DRONECAM
```

4.2 修改配置文件

以使能HY_S0603为例，在HiXuanyuan根目录下，通过make menuconfig命令，打开CONFIG_SENS0_HY_S0603选项。

可以将修改后的文件保存至HiXuanyuan/configs/\$(chip)_smp/dronecam/目录，并重命名为config_\$(chip)_linux_smp_dronecam_demb_hy_s0603_nonescreen_emmc_4096M.conf

说明

当使用的芯片为Hi3516DV500或者Hi3519DV500时，\$(chip)取值为hi3516dv500。当使用SS927V100或者SS928V100时，\$(chip)取值为ss928v100。



4.3 增加对应参数文件

新建source/camera/demo/dronecam/modules/param/iniCfg/\$(chip)/nonescreen/\$(sensor_type)_\$(mem_size)目录

- 其中sensor_type的取值为前文中定义的CFG_SENSOR_TYPE0宏的值；
- mem_size的取值为config.conf文件中CONFIG_MEM_TOTAL_SIZE的值加上M为单位的后缀

以在4096M内存单板上增加HY_S0603的参数为例，新建HiXuanyuan/source/camera/demo/dronecam/modules/param/iniCfg/\$(chip)/nonescreen/hy_s603_4096M目录

需特别注意[vcap.dev]下sensor_type的取值需与配置Sensor信息章节中新增的OT_MAPI_VCAP_SENSOR_HY_S0603枚举值对应。

参数目录中各个ini的含义详情请参考《HiXuanyuan 参考设计开发指南》的4.2 Param 章节。

4.4 新增场景自适应参数

如果需要针对新增的Sensor进行场景自适应参数调整，则需要

HiXuanyuan/source/camera/component/sceneauto/param/的对应芯片目录新增对应的Sensor目录，并修改MAIN_SENSOR_SCENE_PARAM_DIR路径

以新增HY_S0603参数为例，

- 步骤1** 新增HiXuanyuan/source/camera/component/sceneauto/param/\$(chip)/sensor_hy_s0603目录
- 步骤2** 在HiXuanyuan/cmake/build/base_\$(chip)_smp.mak文件中设置
MAIN_SENSOR_SCENE_PARAM_DIR := \$(SOURCE_TREE)/source/camera/
component/sceneauto/param/\$(chip)/sensor_hy_s0603

----结束

4.5 适配 SDK sysconfig

新增Sensor时，需要在smp/a55_linux/source/interdrv/sysconfig进行新Sensor的适配，主要功能为根据不同Sensor进行不同的Sensor Clock选择，详情请参考《MPP 媒体处理软件 FAQ》。

4.6 适配 sysconfig 加载参数

HiXuanyuan代码中维护了独立的load_module文件，位于HiXuanyuan/source/camera/demo/dronecam/rootfs/appfs_priv/komod/目录，以增加HY_S0603为例，将HiXuanyuan/source/camera/demo/dronecam/rootfs/appfs_priv/komod/load_module_\$(chip)_linux_smp文件中，SNS_TYPE0修改为SNS_TYPE0=hy_s0603



5 常见问题

Q: 程序启动后没有报错，但是看不到图像

A: 请先检查Sensor 输入时钟、I2C/SPI 接口管脚复用、输出信号是否正常，检查Sensor 寄存器配置是否正常。

Q: 程序启动正常，但是抓拍的图像颜色不对，或者黑边

A: 检查sensor_interface_cfg_params.c 配置文件中的Bayer 格式、MIPI 裁剪等是否正确。